MCMP117 · Week 15 · Study Guide

Fundamentals of Operating Systems

Final Exam Review & Practice

Fifteen weeks, one exam: from the first batch systems to your own shell scripts.

Instructor: Asst. Prof. Dr. (Ph.D.) Hasan Oğuz
Head of the Electronical Communication Technology Programme, Vocational School
Istanbul Okan University

What you should be able to do for the final exam

1. Explain the central ideas of Weeks 1–7 with the course's own terms: OS views and goals, dual mode, system calls, processes and threads, scheduling, synchronisation, memory management, virtual memory and file systems.
2. Solve the standard numerical problems: utilisation, Amdahl's law, Gantt charts with waiting and turnaround times, address translation, EAT with a TLB and with page faults, page-replacement frame tables, block arithmetic.
3. Write, from memory, the Linux commands of Weeks 9–13: navigation, file operations, permissions, redirection, process and job control.
4. Build one-line pipelines that answer questions about text data (logs, CSV files).
5. Write a short Bash script with an argument check, a loop, a test, arithmetic and a correct exit status.

Contents

1 How to Use This Study Guide 1

2 Theory Recap: Weeks 1–7 2

3 Practical Recap: Weeks 9–14 5

4 Formula and Command Sheet 9

5 The Worked Problems from the Lecture 9

6 Common Mistakes and Exam Strategy 12

7 Practice Final (100 points) 13

8 Practice Final: Worked Solutions 15

9 Course Wrap-up and Where to Go Next 19

1 How to Use This Study Guide

This guide covers the *whole* course. Section 2 recaps the theory weeks 1–7 compactly; it uses exactly the terminology, notation and conventions of the weekly notes. (The Week 8 midterm study guide treats the same weeks at greater length and is still valid.) Section 3 recaps the Linux weeks 9–14 with real terminal output. Section 4 is a one-page formula and command sheet. Section 5 repeats the worked problems from the lecture slides with full solutions, and Section 6 lists common mistakes and an exam strategy. Then comes a complete **Practice Final** (Section 7) with worked solutions (Section 8). The sample paper handed out separately has *different* questions; use it as a timed rehearsal after you have worked through this guide.

1.1 Exam logistics

Items in square brackets are not yet official; the announcement on the course Telegram channel is what counts.

- **Date, time, room:** [to be announced on the course Telegram channel].
- **Duration:** [90] minutes. **Total:** 100 points in six parts (A–F).
- **Bring:** student ID, pens; [a basic, non-programmable calculator — allowed: to be confirmed].
- **Rules:** closed book [to be confirmed]; [a formula sheet is / is not provided — to be confirmed]; phones off.
- **Weight:** [the final counts __ % of the course grade — see the syllabus].

Part	Question type	Weeks	Points*
A	Multiple choice: concepts and commands	1–7, 9–14	20
B	Short answers: explain, compare, list	1–7	12
C	Numerical problems	4–7 (+1, 3)	36
D	Write the command	9–13	14
E	Write the pipeline	12	9
F	Write a short Bash script	14	9

*Point split of the sample paper and of the Practice Final below [the real paper may differ slightly — to be confirmed].

2 Theory Recap: Weeks 1--7

For each week: a short summary, the formulas in a box, and the typical exam questions. If a line here is unfamiliar, go back to that week's notes and redo its review questions.

2.1 Week 1: Introduction and history

A computer follows instructions (input → process → output, program stored in memory); a first working definition of the OS is *the program that starts first and manages everything else*. The history runs from no software (ENIAC, 1945) via batch processing with a resident monitor (GM-NAA I/O, 1956), one OS for a family of machines (IBM System/360, 1964), multiprogramming and time-sharing (CTSS 1961, Multics), UNIX (1969) and the personal computer, to Linux (1991), phones (iPhone 2007, Android 2008) and the cloud. Each step was driven by the cost of an idle processor.

Week 1 formulas

$$U = \frac{t_{\text{run}}}{t_{\text{run}} + t_{\text{setup}}}, \qquad U(n) = 1 - p^n \quad (p = \text{fraction of time a job waits for I/O}).$$

Typical questions: put milestones in order; why batch, why multiprogramming, why time-sharing; compute U, e.g. $t_{\text{run}} = 2$ min, $t_{\text{setup}} = 3$ min gives $U = 2/5 = 0.4$; $p = 0.6$, $n = 3$ gives $U(3) = 1 - 0.216 = 0.784$.

2.2 Week 2: Definitions, goals, components, structures

Two classic views: the OS as an **extended machine** (abstraction, top-down) and as a **resource manager** (time and space multiplexing, bottom-up); its always-resident core is the **kernel**. Design goals: convenience, efficiency, ability to evolve. A computer system has four components: hardware, OS, application programs, users. Devices signal the CPU with **interrupts**. The storage hierarchy runs from registers (≈ 0.3 ns) through cache, RAM (≈ 100 ns) and SSD to HDD (≈ 5 ms); registers, cache and RAM are volatile. **Dual-mode operation**: mode bit 0 = kernel mode (all instructions allowed), 1 = user mode (privileged instructions forbidden); a **system call** switches to kernel mode and costs far more than a function call. Structures: simple (MS-DOS), monolithic (UNIX, Linux), layered (THE), microkernel (MINIX 3, QNX, seL4), modular (loadable kernel modules), hybrid (Windows, macOS, Android). *Typical questions:* the two views; why protection needs hardware; compare monolithic and microkernel (speed versus reliability).

2.3 Week 3: OS services, system calls, processes

Services help the user (user interface, program execution, I/O, file systems, communication, error detection) and the system (resource allocation, accounting, protection and security). A program calls an API function; a thin library wrapper puts the system-call *number* in a register (`rax` on x86-64 Linux) and executes the trap; the kernel uses the number as an index into the **system-call table**. Parameters are passed in registers, in a memory block, or on the stack.

A **process** is a program in execution, with text, data, bss, heap and stack. It moves through the states *new*, *ready*, *running*, *waiting*, *terminated*; transitions: admitted, scheduler dispatch, interrupt, I/O or event wait, I/O or event completion, exit. A waiting process can never go directly to running. The **PCB** stores state, program counter, registers, scheduling, memory and file information. A **context switch** saves one PCB and loads another and is pure overhead. `fork()` creates a copy, `exec()` loads a new program, `wait()` waits for a child.

Week 3 formulas

$$\text{switch overhead} = \frac{s}{q+s}, \qquad n \text{ forks in a loop} \Rightarrow 2^n \text{ processes } (2^n - 1 \text{ created by } \texttt{fork()}).$$

With $s = 0.1$ ms and $q = 4$ ms: $0.1/4.1 = 2.44\,\%$ of the CPU time is spent switching.

2.4 Week 4: Threads and CPU-scheduling basics

A **thread** has its own thread ID, program counter, registers and stack; the threads of a process share code, data, heap and open files. Benefits: responsiveness, resource sharing, economy, scalability. Concurrency means several tasks make progress; parallelism needs several cores. Multithreading models: many-to-one, one-to-one (Linux, Windows), many-to-many. Processes alternate CPU and I/O bursts; the short-term scheduler picks from the ready queue, the dispatcher performs the switch; preemptive schedulers may interrupt a running process.

Week 4 formulas

$$\text{speedup}(N) \le \frac{1}{S + \dfrac{1-S}{N}}, \qquad \lim_{N\to\infty} = \frac{1}{S};$$

$$T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}, \qquad T_{\text{waiting}} = T_{\text{turnaround}} - T_{\text{burst}},$$

$$T_{\text{response}} = T_{\text{first run}} - T_{\text{arrival}};$$

$$\text{CPU utilisation} = \frac{B}{T}, \qquad \text{throughput} = \frac{n}{T}.$$

Typical questions: what threads share; compare the models; Amdahl ($S = 0.2$, $N = 4$: $1/(0.2+0.2) = 2.5$, never more than 5); the criteria (maximise utilisation and throughput, minimise turnaround, waiting and response time).

2.5 Week 5: Scheduling algorithms and synchronisation

FCFS (convoy effect), **SJF** (optimal average waiting time when all jobs are ready together), **SRTF** (its preemptive version), **Priority** (smaller number = higher priority; starvation, cured by **aging**), **Round Robin** (quantum q; wait between turns at most $(n-1)q$; q too large $\to$ FCFS, too small $\to$ switching overhead).

Scheduling conventions of this course --- state them in the exam

Context-switch time is 0; ties are broken by arrival time, then by process name; a *smaller* priority number means a *higher* priority; in Round Robin, a process that arrives at the same instant as a quantum expires enters the ready queue *before* the preempted process.

A **race condition** arises when concurrent threads access shared data, at least one writes, and the result depends on timing (`counter++` is load–add–store). A solution to the **critical-section problem** must guarantee *mutual exclusion*, *progress* and *bounded waiting*. Mutex locks (`acquire/release`) and semaphores (`wait(S)`: wait while $S \le 0$, then $S{-}{-}$; `signal(S)`: $S{+}{+}$; POSIX `sem_wait/sem_post`) are built on

atomic hardware instructions. The bounded buffer uses `empty`, `full` and `mutex`; locks taken in inconsistent orders can deadlock.

Week 5 formula

$$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\,\tau_n, \qquad 0 \le \alpha \le 1.$$

2.6 Week 6: Memory management

Address binding at compile, load or execution time; modern systems bind at execution time, so **logical** and **physical** addresses differ and the **MMU** translates. Base and limit: if $0 \le a <$ limit then $a_{\text{phys}} = \text{base} + a$, otherwise trap. Contiguous allocation with first-fit (first hole big enough), best-fit (smallest big enough), worst-fit (largest) suffers from **external fragmentation** (50-percent rule: up to one third of memory lost); compaction helps but is expensive. **Paging** splits logical memory into pages and physical memory into equal frames: no external fragmentation, only **internal fragmentation** (half a page per process on average). A **TLB** caches page-table entries. **Segmentation** translates $\langle s, d\rangle$ with a base and a limit per segment.

Week 6 formulas

With pages of 2^n bytes:

$$p = \left\lfloor \frac{a}{2^n} \right\rfloor, \qquad d = a \bmod 2^n, \qquad a_{\text{phys}} = f \cdot 2^n + d;$$

$$\text{EAT} = \alpha\,(t_{\text{TLB}} + t_m) + (1-\alpha)\,(t_{\text{TLB}} + 2t_m) = t_{\text{TLB}} + (2-\alpha)\,t_m \quad \text{(our convention)}.$$

Page-table size = number of pages $\times$ entry size, e.g. $2^{20} \times 4$ B $= 4$ MB for 32-bit addresses and 4 KB pages.

2.7 Week 7: Virtual memory and file systems

Demand paging loads a page on first use; the valid–invalid bit marks pages that are not in memory; touching one causes a **page fault**: trap, locate on disk, read into a free frame, update the table, restart the instruction. Copy-on-write makes `fork()` cheap. When no frame is free, a replacement algorithm chooses a victim: **FIFO** (oldest page; Belady's anomaly possible), **OPT** (used farthest in the future; optimal but needs the future), **LRU** (least recently used; approximated by the clock/second-chance algorithm). **Thrashing**: processes have fewer frames than their working sets and spend their time paging. Files are named data with attributes (Linux: the inode); access is sequential, direct or indexed. Directories: single-level, two-level, tree, acyclic graph (hard and symbolic links), general graph.

Week 7 formulas

$$\text{EAT} = (1-p)\,t_m + p\,t_{\text{fault}}; \qquad \text{byte} = r \times \text{record size}, \quad \text{block} = \left\lfloor \frac{\text{byte}}{B} \right\rfloor, \quad \text{offset} = \text{byte} \bmod B.$$

3 Practical Recap: Weeks 9--14

About the outputs. Every terminal session in this section is real output, captured as an ordinary user in a freshly created `~/lab` folder (run `bash mcmp117_lab_setup.sh` first; the script is in `code/`). The commands are in `code/demo_recap.sh`, `code/demo_worked.sh` and `code/answers_*.sh`, the outputs in `code/out_*.txt`. On your machine, dates, sizes, PIDs and user names will differ, and the order of `ls` and `sort` output can depend on the locale; we used `LC_ALL=C.UTF-8`.

3.1 Week 9: Shell basics and navigation

The *terminal* is the window, the *shell* (bash) is a system program that reads commands, and the kernel does the work through system calls (Week 2). A command has the form `command -options arguments`. `pwd` prints the current directory, `ls` lists (`-l` long, `-a` hidden, `-h` human-readable sizes, `-R` recursive), `cd` changes directory. Absolute paths start at `/`; relative paths start at the current directory; `~` is home, `.` here, `..` the parent, `cd -` goes back. Important directories (FHS): `/home`, `/etc`, `/bin`, `/usr`, `/var`, `/tmp`, `/dev`, `/proc`. Help: `man` (`q` quits, `/word` searches), `--help`. Tab completion, `history`, `!!`, Ctrl+C, Ctrl+L.

```
student@okan-lab:~/lab$ pwd
/home/student/lab
student@okan-lab:~/lab$ ls
README.txt  backup  data  docs  logs  photos  projects  scripts  src
student@okan-lab:~/lab$ cd projects/alpha/src
student@okan-lab:~/lab/projects/alpha/src$ pwd
/home/student/lab/projects/alpha/src
student@okan-lab:~/lab/projects/alpha/src$ cd ../../beta/docs
student@okan-lab:~/lab/projects/beta/docs$ pwd
/home/student/lab/projects/beta/docs
student@okan-lab:~/lab/projects/beta/docs$ cd ~/lab
```

3.2 Week 10: File and directory operations

`mkdir -p`, `touch`, `cp` (`-r -i -v`), `mv` (rename or move), `rm` (`-r -i`; **there is no recycle bin**: check the path before you press Enter), `rmdir`. Wildcards `*` (any string), `?` (exactly one character), `[abc]`, `[0-9]`, and brace expansion `{a,b}`, `{1..5}` are expanded *by the shell* before the command runs. Quote names with spaces. Viewing: `cat`, `less`, `head`, `tail -n/-f`, `wc`, `file`, `stat`; sizes `du -sh`, `df -h`; links `ln` (hard) and `ln -s` (symbolic), inode numbers with `ls -i` (Week 7).

```
student@okan-lab:~/lab$ ls photos/img00?.jpg
photos/img001.jpg  photos/img004.jpg  photos/img007.jpg
photos/img002.jpg  photos/img005.jpg  photos/img008.jpg
photos/img003.jpg  photos/img006.jpg  photos/img009.jpg
student@okan-lab:~/lab$ echo photos/*.{png,JPG}
photos/img013.png photos/IMG_014.JPG
student@okan-lab:~/lab$ mkdir -p backup/{2025,2026}
student@okan-lab:~/lab$ cp docs/notes_0*.txt backup/2026/
student@okan-lab:~/lab$ ls backup/2026
notes_01.txt  notes_02.txt  notes_03.txt
student@okan-lab:~/lab$ wc -l data/students.csv docs/syllabus.txt
  41 data/students.csv
  15 docs/syllabus.txt
  56 total
student@okan-lab:~/lab$ ln -s ../logs/access.log docs/latest.log
student@okan-lab:~/lab$ ls -l docs/latest.log
lrwxrwxrwx 1 student student 18 Oct 12 08:50 docs/latest.log -> ../logs/access.log
```

3.3 Week 11: Permissions, ownership and users

The mode string `-rwxr-x---` has a type character and three triplets for owner (u), group (g) and others (o). For files r/w/x mean read/write/execute; for directories r lists, w creates and deletes entries, x enters (`cd`). Octal: $r = 4, w = 2, x = 1$, so `rwxr-x---` $= 750$ ($7 = 4+2+1, 5 = 4+1, 0$). `chmod` symbolic (`u+x`, `go-w`, `a=r`) or octal. The **umask** *removes* bits from the requested mode (666 for files, 777 for directories): new mode $=$ requested mode AND NOT umask (bitwise, not a subtraction); umask 002 gives $664/775$, umask 027 gives $640/750$. `chown`/`chgrp` need root; `sudo` runs one command as root — think before you type it. Never "fix" a permission problem with `chmod 777`: it lets every user change the file.

```
student@okan-lab:~/lab$ id
uid=1015(student) gid=1017(student) groups=1017(student)
student@okan-lab:~/lab$ touch scripts/run.sh
student@okan-lab:~/lab$ ls -l scripts/run.sh
-rw-rw-r-- 1 student student 0 Oct 12 08:50 scripts/run.sh
student@okan-lab:~/lab$ chmod 750 scripts/run.sh
student@okan-lab:~/lab$ ls -l scripts/run.sh
-rwxr-x--- 1 student student 0 Oct 12 08:50 scripts/run.sh
student@okan-lab:~/lab$ chmod g-x,o+r scripts/run.sh
student@okan-lab:~/lab$ ls -l scripts/run.sh
-rwxr--r-- 1 student student 0 Oct 12 08:50 scripts/run.sh
student@okan-lab:~/lab$ umask
0002
```

3.4 Week 12: Text processing, redirection and pipes

Every process starts with three open file descriptors: 0 stdin, 1 stdout, 2 stderr (Weeks 2–3: `write(1, …)`). Redirection: `>` overwrite, `>>` append, `<` input, `2>` errors, `2>&1` errors to where stdout goes, `&>` both, `/dev/null` discards. A pipe `a | b` connects stdout of `a` to stdin of `b`; the shell creates both processes (Week 3). Filters: `grep` (`-i -n -v -c -r -w`; regex `^ $ . [ ] *`), `sort` (`-n -r -t -k`), `uniq` (`-c`; needs sorted input), `cut` (`-d -f`), `wc`, `tr`, `head`, `tail`, `tee`, `find` (`-name -type -size -mtime`), `xargs`.

```
student@okan-lab:~/lab$ grep -c ERROR logs/system.log
13
student@okan-lab:~/lab$ grep -n ERROR logs/system.log | head -3
13:2026-10-12 09:12:00 ERROR failed to mount /dev/sdb1
17:2026-10-12 09:56:00 ERROR failed to mount /dev/sdb1
18:2026-10-12 09:07:00 ERROR failed to mount /dev/sdb1
student@okan-lab:~/lab$ cut -d' ' -f9 logs/access.log | sort | uniq -c | sort -rn
    148 200
     57 404
     34 304
     24 500
     20 301
     17 403
student@okan-lab:~/lab$ ls nosuchfile 2> errors.txt
student@okan-lab:~/lab$ cat errors.txt
ls: cannot access 'nosuchfile': No such file or directory
```

In `logs/access.log` the space-separated fields are: 1 IP address, 4–5 date and time zone, 6–8 request (method, page, protocol), 9 status code, 10 bytes. In `data/students.csv` (header `id,name,department,midterm,final`) the comma-separated fields are 1–5.

3.5 Week 13: Processes and job control

`ps`, `ps -ef`, `ps aux` (columns PID, PPID, STAT, %CPU, %MEM), `pstree`, `top`/`htop`, `/proc/<pid>/status`. STAT shows the Week 3 states: R running or ready, S sleeping (waiting), T stopped, Z zombie. Jobs: `cmd &` runs in the background, `jobs` lists, `fg`/`bg` move, Ctrl+Z stops (SIGTSTP), Ctrl+C interrupts (SIGINT). Signals: `kill PID` sends SIGTERM (15, "please stop"), `kill -9` SIGKILL (cannot be caught or ignored — the process gets no chance to clean up: use it only when SIGTERM fails), SIGSTOP (19) and SIGCONT (18); `pkill`, `killall`. `$$` is the PID of the shell, `$!` of the last background job. `nice`/`renice`: values -20 (highest priority) to 19; only root may lower them (Week 5).

```
student@okan-lab:~/lab$ sleep 600 &
student@okan-lab:~/lab$ echo $!
23441
student@okan-lab:~/lab$ jobs
[1]+  Running                 sleep 600 &
student@okan-lab:~/lab$ ps -o pid,ppid,stat,cmd
  PID  PPID STAT CMD
23361 23360 Ss+  bash /home/student/w15/demo_recap.sh
23441 23361 S+   sleep 600
23445 23361 R+   ps -o pid,ppid,stat,cmd
student@okan-lab:~/lab$ kill %1
student@okan-lab:~/lab$ kill -l TERM KILL INT STOP CONT
15
9
2
19
18
```

The first `ps` line is the shell that ran our commands (here a script; in your terminal you will see `bash`). In an interactive terminal the shell also reports `[1]+ Terminated sleep 600` at the next prompt.

3.6 Week 14: Shell scripting

A script starts with the shebang `#!/bin/bash`; run it with `chmod +x s.sh; ./s.sh` or `bash s.sh`; debug with `bash -x`. Variables: x=5 (no spaces around =), `"$x"` expands, `'$x'` does not; command substitution `$(cmd)`; integer arithmetic `$((x + 1))`. Positional parameters `$0 $1 $# $@`; `$?` is the exit status of the last command, `exit n` sets the script's own. Tests `[ … ]` (spaces inside the brackets!): numbers `-eq -ne -lt -le -gt -ge`, strings `= != -z`, files `-f -d -e`. Control: `if/elif/else/fi`, `for … in …; do …; done`, `while`, functions. Week 14 built `backup.sh` and `report.sh`.

```
student@okan-lab:~/lab$ name=student; echo "Hello $name"
Hello student
student@okan-lab:~/lab$ echo 'Hello $name'
Hello $name
student@okan-lab:~/lab$ echo "2^10 = $((2 ** 10))"
2^10 = 1024
student@okan-lab:~/lab$ n=$(wc -l < data/students.csv); echo "students.csv has $n lines"
students.csv has 41 lines
student@okan-lab:~/lab$ [ -d logs ] && echo "logs is a directory"
logs is a directory
student@okan-lab:~/lab$ for c in 1 2 3; do echo "item $c"; done
item 1
item 2
item 3
```

4 Formula and Command Sheet

Week	Quantity	Formula
1	Batch utilisation	$U = t_{\text{run}}/(t_{\text{run}} + t_{\text{setup}})$
1	Multiprogramming	$U(n) = 1 - p^n$
3	Switch overhead; forks	$s/(q+s)$; n forks in a loop $\Rightarrow 2^n$ processes
4	Amdahl's law	speedup $\leq 1/\big(S + (1-S)/N\big)$, limit $1/S$
4	Times	$T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}$; $T_{\text{waiting}} = T_{\text{turnaround}} - T_{\text{burst}}$; $T_{\text{response}} = T_{\text{first run}} - T_{\text{arrival}}$
4	Utilisation, throughput	B/T, n/T
5	Burst prediction; RR	$\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$; wait between turns $\leq (n-1)q$
6	Base/limit	$0 \leq a <$ limit $\Rightarrow a_{\text{phys}} =$ base $+ a$, else trap
6	Paging	$p = \lfloor a/2^n \rfloor$, $d = a \bmod 2^n$, $a_{\text{phys}} = f \cdot 2^n + d$
6	TLB	EAT $= t_{\text{TLB}} + (2-\alpha)\,t_m$ (simplified, $t_{\text{TLB}} = 0$: $\alpha t_m + (1-\alpha)2t_m$)
7	Page faults	EAT $= (1-p)\,t_m + p\,t_{\text{fault}}$
7	Direct access	byte $= r\times$ size; block $= \lfloor$byte$/B\rfloor$; offset $=$ byte $\bmod B$

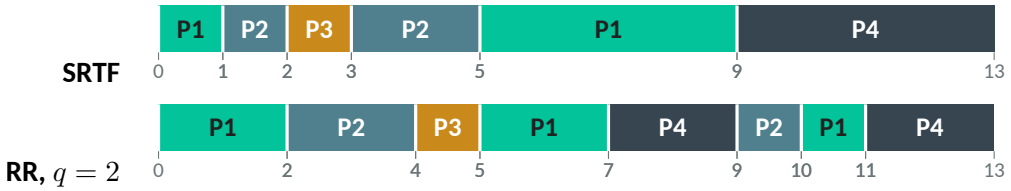
Task	Command	Task	Command
Where am I?	`pwd`	Find lines	`grep -n -i -v -c -w`
List long, hidden	`ls -la  (-h)`	Field k	`cut -d, -f3`
Change directory	`cd ~  cd -  cd ..`	Sort	`sort -n -r -t, -k5`
Make folders	`mkdir -p a/b`	Count duplicates	`sort | uniq -c`
Copy / move	`cp -r a b  mv a b`	Skip header	`tail -n +2 f`
Delete	`rm -i f  rm -r dir`	Find files	`find . -name "*.c"`
Links	`ln f h  ln -s f s`	Redirect	`> >> < 2> 2>&1 &>`
Look at a file	`cat less head tail`	Processes	`ps aux  ps -ef  top`
Count, sizes	`wc -l  du -sh  df -h`	Jobs	`cmd &  jobs  fg %1  bg`
Permissions	`chmod 640 f  chmod u+x f`	Signals	`kill PID  kill -9 PID`
Users	`whoami  id  groups`	Priority	`nice -n 10 cmd`
Owner (root)	`sudo chown u:g f`	Script	`#!/bin/bash  $1 $# $?`
Help	`man ls  ls --help`	Arithmetic	`$((x + 1))  $(cmd)`

5 The Worked Problems from the Lecture

All numbers below are computed by `code/theory_check.py` (output in `code/out_theory_check.txt`).

5.1 Problem 1: SRTF and Round Robin

P1 (arrival 0, burst 5), P2 (1, 3), P3 (2, 1), P4 (3, 4), times in ms.



SRTF: at $t = 1$ P2 (3) preempts P1 (4 left); at $t = 2$ P3 (1) preempts P2; P2 finishes at 5; then P1 and P4 both need 4 ms and the tie goes to P1 (earlier arrival). Waiting times $4, 1, 0, 6$: average $11/4 = 2.75$ ms; turnaround $9, 4, 1, 10$: average 6 ms. RR: at $t = 2$ the queue is P2, P3, P1 (P3 arrived exactly at 2 and enters before the preempted P1). Waiting $6, 6, 2, 6$: average 5 ms; turnaround $11, 9, 3, 10$: average 8.25 ms; response $0, 1, 2, 4$: average 1.75 ms (SRTF: 1.5 ms).

5.2 Problem 2: address translation and TLB

Pages of 1 KB $= 2^{10}$ bytes; page table $0 \to 5$, $1 \to 2$, $2 \to 7$, $3 \to 1$. Logical address 2500: $p = \lfloor 2500/1024 \rfloor = 2$, $d = 2500 - 2048 = 452$, frame 7, $a_{\text{phys}} = 7 \times 1024 + 452 = 7620$. With $t_{\text{TLB}} = 10$ ns, $t_m = 100$ ns, $\alpha = 0.9$: EAT $= 10 + 1.1 \times 100 = 120$ ns ($= 0.9 \times 110 + 0.1 \times 210$). For EAT $= 115$ ns: $10 + (2 - \alpha)\,100 = 115 \Rightarrow \alpha = 0.95$.

5.3 Problem 3: page replacement

Reference string $3, 5, 1, 0, 3, 1, 0, 3, 1, 4, 1, 0$ with 3 frames: FIFO 8, LRU 7, OPT 5 faults. The LRU table (bold = page just loaded):

ref	**3**	**5**	**1**	**0**	**3**	**1**	**0**	**3**	**1**	**4**	**1**	**0**
fr. 0	**3**	3	3	**0**	0	0	0	0	0	**4**	4	4
fr. 1		**5**	5	5	**3**	3	3	3	3	3	3	**0**
fr. 2			**1**	1	1	1	1	1	1	1	1	1
fault	**F**	**F**	**F**	**F**	**F**					**F**		**F**

OPT at the fourth reference (0) evicts 5, which is never used again, and at reference 4 evicts 3, also never needed again: 5 faults, three of them compulsory.

5.4 Problem 4: short calculations

- $t_m = 100$ ns, $t_{\text{fault}} = 10$ ms $= 10^7$ ns, $p = 0.0002$: EAT $= 0.9998 \times 100 + 0.0002 \times 10^7 = 99.98 + 2000 = 2099.98$ ns, 21 times slower than RAM. Slow-down below $10\,\%$: $p < 10/9\,999\,900 \approx 1.0 \times 10^{-6}$.
- Record 75, records of 128 B, blocks of 4096 B: byte 9600, block $\lfloor 9600/4096 \rfloor = 2$, offset $9600 - 8192 = 1408$; the last byte 9727 is still in block 2.
- $U(3) = 1 - 0.6^3 = 0.784$; batch $2/(2 + 3) = 0.4$; overhead $0.1/4.1 = 2.44\,\%$; Amdahl $1/(0.2 + 0.8/4) = 2.5$.

5.5 Problems 5--7: command, pipeline, script

Permissions. `rwx r-x ---` $= 750$, so `chmod 750 report.sh`. After `umask 027` the bits 0, 2, 7 are removed from $666/777$: new files 640, new directories 750.

```
student@okan-lab:~/lab$ touch report.sh
student@okan-lab:~/lab$ ls -l report.sh
-rw-rw-r-- 1 student student 0 Oct 12 08:50 report.sh
student@okan-lab:~/lab$ chmod 750 report.sh
student@okan-lab:~/lab$ ls -l report.sh
-rwxr-x--- 1 student student 0 Oct 12 08:50 report.sh
student@okan-lab:~/lab$ umask 027
student@okan-lab:~/lab$ touch new.txt; mkdir newdir
student@okan-lab:~/lab$ ls -ld new.txt newdir
-rw-r----- 1 student student    0 Oct 12 08:50 new.txt
drwxr-x--- 2 student student 4096 Oct 12 08:50 newdir
```

Pipelines on `access.log`. The top-N pattern `cut | sort | uniq -c | sort -rn | head`. For the 404 count, cut the status field first: `404` could also occur inside a byte count.

```
student@okan-lab:~/lab$ cut -d' ' -f1 logs/access.log | sort | uniq -c | sort -rn | head -3
     58 10.0.2.17
     56 10.0.4.31
     52 10.0.6.45
student@okan-lab:~/lab$ cut -d' ' -f9 logs/access.log | grep -c '^404$'
57
student@okan-lab:~/lab$ grep ' 404 ' logs/access.log | cut -d' ' -f7 | sort | uniq -c | sort -
    rn | head -1
      9 /images/logo.png
```

Script `dept_stats.sh`. Note the integer division in `$((sum / count))`: $1072/17 = 63.06$ is printed as 63.

```bash
#!/bin/bash
# dept_stats.sh -- number of students and average final grade of ONE department
# usage: ./dept_stats.sh DEPARTMENT        (run inside ~/lab)
if [ $# -ne 1 ]; then
    echo "usage: $0 DEPARTMENT" >&2
    exit 1
fi
dept=$1
count=0
sum=0
for f in $(grep ",$dept," data/students.csv | cut -d, -f5); do
    count=$((count + 1))
    sum=$((sum + f))
done
if [ $count -eq 0 ]; then
    echo "no students in $dept"
    exit 2
fi
echo "$dept: $count students, average final $((sum / count))"
```

```
student@okan-lab:~/lab$ chmod u+x scripts/dept_stats.sh
student@okan-lab:~/lab$ ./scripts/dept_stats.sh ECT
ECT: 17 students, average final 63
student@okan-lab:~/lab$ ./scripts/dept_stats.sh CS
CS: 10 students, average final 65
student@okan-lab:~/lab$ ./scripts/dept_stats.sh
usage: ./scripts/dept_stats.sh DEPARTMENT
student@okan-lab:~/lab$ echo $?
1
```

6 Common Mistakes and Exam Strategy

Waiting vs. turnaround Waiting $=$ turnaround $-$ burst; turnaround starts at the *arrival*, not at 0.

Round Robin queue A process arriving at the moment a quantum expires enters before the preempted process.

Units 8 ms $= 8\,000\,000$ ns. Convert first, then substitute.

TLB miss A miss costs two memory accesses (page table and data) plus the TLB lookup.

LRU vs. OPT LRU looks backwards, OPT looks forwards.

Block arithmetic Records and blocks are numbered from 0; check whether the last byte crosses into the next block.

`uniq` without `sort` `uniq -c` merges only *adjacent* equal lines.

Spaces x=5, not `x = 5` (“command not found”); `[ $a -gt 3 ]`, not `[$a -gt 3]`.

Quotes `"$x"` expands, `'$x'` does not; quote names with spaces: `"final report.txt"`.

> vs. >> > overwrites, >> appends, 2> redirects only errors.

Too strong tools `chmod 777`, `rm -r`, `kill -9`: too much permission, no undo, no clean-up.

Exam strategy

- **Before:** redo the review questions of Weeks 1–7 and the lab sheets of Weeks 9–14; solve the Practice Final below on paper with a timer, then the sample paper; type every command of the sheet in `~/lab`.
- **During:** read the whole paper first (5 min); start with what you know best; about one minute per point, so do Parts A and D quickly and spend the time on Part C; write formula → numbers → result with unit; state the conventions you use; never leave a question empty.

7 Practice Final (100 points)

Solve this paper on your own, in [90] minutes, before you read the solutions in Section 8. Commands refer to the folder `~/lab` created by `mcmp117_lab_setup.sh`; assume your current directory is `~/lab` unless a question says otherwise.

Part A --- Multiple choice ($10 \times 2 = 20$ points)

A1. Which of these is *not* one of the three OS design goals of Week 2? (a) convenience (b) efficiency (c) ability to evolve (d) maximum hardware cost

A2. In kernel mode the mode bit is (a) 0 (b) 1 (c) 2 (d) undefined

A3. A waiting process whose I/O completes moves to the state (a) running (b) ready (c) new (d) terminated

A4. Which of these do the threads of one process *share*? (a) stack (b) registers (c) heap (d) program counter

A5. Which algorithm can show Belady's anomaly? (a) OPT (b) LRU (c) FIFO (d) none

A6. A TLB caches (a) disk blocks (b) page-table entries (c) PCBs (d) file names

A7. A directory has the mode `drwxr-x---`. A member of its group can (a) create files in it (b) list and enter it (c) only enter it (d) nothing

A8. Which redirection *appends* standard output to a file? (a) > (b) >> (c) 2> (d) <

A9. Which signal can a process neither catch nor ignore? (a) SIGTERM (b) SIGINT (c) SIGKILL (d) SIGTSTP

A10. In a Bash script, `$#` is (a) the script name (b) the PID (c) the number of arguments (d) the last exit status

Part B --- Short answers ($3 \times 4 = 12$ points)

B1. What is the difference between a program and a process? Name the five process states and the event that moves a process from running to waiting. **[4 pt]**

B2. Explain internal and external fragmentation. Which one occurs with paging, and why not the other? **[4 pt]**

B3. `notes.txt` has a hard link `h.txt` and a symbolic link `s.txt`. What happens to each after `rm notes.txt`? Explain with inodes. **[4 pt]**

Part C --- Numerical problems (36 points)

C1. Processes P1 (arrival 0, burst 8), P2 (1, 4), P3 (2, 9), P4 (3, 5), times in ms. Draw the Gantt charts and compute the average waiting and turnaround times for (a) SJF (non-preemptive), (b) SRTF, (c) Round Robin with $q = 3$. Use the course conventions. **[12 pt]**

C2. (a) Pages are 4 KB. Page 5 is in frame 9. Translate the logical address `0x5C21` (give the result in hexadecimal and decimal). (b) How large is the page table for 32-bit addresses, 8 KB pages and 4-byte entries? (c) $t_{\text{TLB}} = 20$ ns, $t_m = 80$ ns, $\alpha = 0.95$: compute the EAT (our convention). (d) Which hit ratio gives an EAT of 110 ns? **[8 pt]**

C3. Reference string $2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2$ with 3 frames: draw the frame tables and count the page faults for FIFO, OPT and LRU. **[8 pt]**

C4. (a) $t_m = 150$ ns, $t_{\text{fault}} = 6$ ms, $p = 0.0001$: compute the EAT. What is the largest p for a slow-down below $5\,\%$? (b) Records are 200 B, blocks 1024 B. In which block and at which offset does record 30 start? Does it fit in one block? (c) Holes of 300, 600, 350, 200, 750, 125 KB (in this order); requests 115, 500, 358, 200, 375 KB arrive in this order. Where does each request go with first-fit, best-fit and worst-fit? **[8 pt]**

Part D --- Write the command ($7 \times 2 = 14$ points)

D1. You are in the directory `~/lab/projects/alpha/src`.
With *one* `cd` and a *relative* path, go to `~/lab/projects/beta/docs`.

D2. Create the directories `archive/2025` and `archive/2026` with one command.

D3. Copy all files ending in `.txt` from `docs/` into `backup/`.

D4. Show the last three lines of `logs/system.log`.

D5. Make `docs/syllabus.txt` readable and writable by its owner only (octal).

D6. Start `sleep 600` in the background, list your jobs, then stop it politely with a signal.

D7. List all files ending in `.c` anywhere below `projects`.

Part E --- Write the pipeline ($3 \times 3 = 9$ points)

E1. How many lines of `logs/system.log` contain `ERROR`?

E2. For `logs/access.log`, print how many requests got each status code, most frequent first.

E3. For `data/students.csv`, print the number of students per department, largest first (skip the header line).

Part F --- Write a script (9 points)

Write `count_lines.sh` that takes one argument, a directory. If the argument is missing or is not a directory, print `usage: …` to standard error and exit with status 1. Otherwise print, for every regular file in the directory, its name and number of lines, and finally the total.

8 Practice Final: Worked Solutions

Part A

A1 (d), A2 (a), A3 (b), A4 (c), A5 (c), A6 (b), A7 (b): r lets the group list, x lets it enter, without w it cannot create files; A8 (b), A9 (c), A10 (c).

Part B

B1. A program is a passive file on disk; a process is a program in execution, with its own memory (text, data, bss, heap, stack), registers and PCB; one program can run as several processes. States: new, ready, running, waiting, terminated. Running $\rightarrow$ waiting happens when the process makes a system call that must wait (I/O, `wait()` for a child, `sleep`).

B2. External fragmentation: free memory is split into holes between allocated blocks, none big enough although the total would be. Internal fragmentation: memory inside an allocated block that is not used. Paging has only internal fragmentation (on average half a page per process, in the last page), because any free frame can hold any page, so no hole is ever too small.

B3. `h.txt` is a second directory entry for the same inode; `rm` removes only the name `notes.txt` and lowers the link count from 2 to 1, so the data is still readable through `h.txt`. `s.txt` has its own inode and only contains the path `notes.txt`; that path no longer exists, so `s.txt` dangles ("No such file or directory").

Part C

C1 (a) SJF. At $t = 0$ only P1 is ready and runs to 8; then P2 (4), P4 (5), P3 (9).



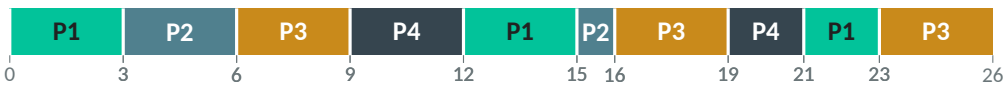
Waiting: P1 0, P2 $8 - 1 = 7$, P3 $17 - 2 = 15$, P4 $12 - 3 = 9$; average $31/4 = 7.75$ ms. Turnaround $8, 11, 24, 14$: average $57/4 = 14.25$ ms.

(b) SRTF. At $t = 1$ P2 (4) preempts P1 $(7$ left); at $t = 5$ P4 (5) runs before P1 (7); P3 (9) last.



Waiting: P1 $17 - 0 - 8 = 9$, P2 0, P3 15, P4 2; average $26/4 = 6.5$ ms. Turnaround $17, 4, 24, 7$: average $52/4 = 13$ ms.

(c) RR, $q = 3$**.** Queue at $t = 3$: P2, P3, P4 (all arrived by 3; P4 arrived exactly at 3 and enters before the preempted P1), P1.



Completion $23, 16, 26, 21$; turnaround $23, 15, 24, 18$: average $80/4 = 20$ ms. Waiting $15, 11, 15, 13$: average $54/4 = 13.5$ ms (response: average 3 ms — RR is worst for waiting here, best for response).

C2. (a) 4 KB $= 2^{12}$: the last three hex digits are the offset. $p =$ `0x5`, $d =$ `0xC21` $= 3105$; frame 9: $a_{\text{phys}} =$ `0x9C21` $= 9 \times 4096 + 3105 = 39\,969$. (b) $2^{32}/2^{13} = 2^{19} = 524\,288$ entries $\times$ 4 B $= 2^{21}$ B $= 2$ MB. (c) EAT $= 20 + (2 - 0.95) \times 80 = 20 + 84 = 104$ ns. (d) $20 + (2 - \alpha)\,80 = 110 \Rightarrow 2 - \alpha = 1.125 \Rightarrow \alpha = 0.875$.

C3. FIFO 9, OPT 6, LRU 7 faults (bold = page just loaded).

FIFO

ref	**2**	**3**	**2**	**1**	**5**	**2**	**4**	**5**	**3**	**2**	**5**	**2**
fr. 0	**2**	2	2	2	**5**	5	5	5	**3**	3	3	3
fr. 1		**3**	3	3	3	**2**	2	2	2	2	**5**	5
fr. 2				**1**	1	1	**4**	4	4	4	4	**2**
fault	**F**	**F**		**F**	**F**	**F**	**F**		**F**		**F**	**F**

OPT

ref	**2**	**3**	**2**	**1**	**5**	**2**	**4**	**5**	**3**	**2**	**5**	**2**
fr. 0	**2**	2	2	2	2	2	**4**	4	4	**2**	2	2
fr. 1		**3**	3	3	3	3	3	3	3	3	3	3
fr. 2				**1**	**5**	5	5	5	5	5	5	5
fault	**F**	**F**		**F**	**F**		**F**			**F**		

LRU

ref	**2**	**3**	**2**	**1**	**5**	**2**	**4**	**5**	**3**	**2**	**5**	**2**
fr. 0	**2**	2	2	2	2	2	2	2	**3**	3	3	3
fr. 1		**3**	3	3	**5**	5	5	5	5	5	5	5
fr. 2				**1**	1	1	**4**	4	4	**2**	2	2
fault	**F**	**F**		**F**	**F**		**F**		**F**	**F**		

Key decisions: OPT at reference 5 evicts 1 (never used again), at 4 evicts 2 (used again latest of $2, 3, 5$: positions $10, 9, 8$ — 2 is needed at position 10, 3 at 9, 5 at 8, so 2 goes), at the second 2 evicts 4 (4 and 3 are never used again; either choice gives the same count). LRU at 5 evicts 3 (least recently used), at 4 evicts 1.

C4. (a) EAT $= 0.9999 \times 150 + 0.0001 \times 6\,000\,000 = 149.985 + 600 = 749.985$ ns, five times slower than RAM. Slow-down below $5\,\%$: $150 + p\,(6\,000\,000 - 150) < 157.5 \Rightarrow p < 7.5/5\,999\,850 \approx 1.25 \times 10^{-6}$. (b) Byte $30 \times 200 = 6000$; block $\lfloor 6000/1024 \rfloor = 5$; offset $6000 - 5120 = 880$. The last byte is 6199, in block $\lfloor 6199/1024 \rfloor = 6$: the record crosses a block boundary and needs two block reads. (c)

	115	**500**	**358**	**200**	**375**
First-fit	300 (hole 1)	600 (hole 2)	750 (hole 5)	350 (hole 3)	392 left of hole 5
Best-fit	125 (hole 6)	600 (hole 2)	750 (hole 5)	200 (hole 4)	392 left of hole 5
Worst-fit	750 (hole 5)	635 left of hole 5	600 (hole 2)	350 (hole 3)	**must wait**

After worst-fit the holes are $300, 242, 150, 200, 135, 125$ KB: none can hold 375 KB, although 1152 KB are free in total — external fragmentation.

Part D

Real output of each answer (`code/answers_practice.sh`):

```
student@okan-lab:~/lab$ cd projects/alpha/src
student@okan-lab:~/lab/projects/alpha/src$ cd ../../beta/docs
student@okan-lab:~/lab/projects/beta/docs$ pwd
/home/student/lab/projects/beta/docs
```

```
student@okan-lab:~/lab$ mkdir -p archive/{2025,2026}
student@okan-lab:~/lab$ ls archive
2025  2026
```

```
student@okan-lab:~/lab$ cp docs/*.txt backup/
student@okan-lab:~/lab$ ls backup
'final report.txt'   notes_01.txt   notes_03.txt
 linux_quotes.txt    notes_02.txt   syllabus.txt
student@okan-lab:~/lab$ ls backup | wc -l
6
```

The wildcard also matched `final report.txt`: the shell passes the name with its space as *one* argument, so `cp` has no problem with it.

```
student@okan-lab:~/lab$ tail -n 3 logs/system.log
2026-10-12 14:27:00 INFO  user login
2026-10-12 14:38:00 INFO  backup finished
2026-10-12 14:49:00 INFO  network link up
```

```
student@okan-lab:~/lab$ chmod 600 docs/syllabus.txt
student@okan-lab:~/lab$ ls -l docs/syllabus.txt
-rw------- 1 student student 578 Oct 12 08:50 docs/syllabus.txt
```

```
student@okan-lab:~/lab$ sleep 600 &
student@okan-lab:~/lab$ jobs
[1]+  Running                 sleep 600 &
student@okan-lab:~/lab$ kill %1
student@okan-lab:~/lab$ jobs
```

`kill %1` sends SIGTERM (15); `kill -15 %1` or `kill PID` are also correct, `kill -9` is not “polite”.

```
student@okan-lab:~/lab$ find projects -name "*.c"
projects/beta/src/main.c
projects/alpha/src/main.c
```

Quote the pattern (`"*.c"`), so that `find` and not the shell expands it.

Part E

```
student@okan-lab:~/lab$ grep -c ERROR logs/system.log
13
```

(`grep ERROR logs/system.log | wc -l` is also correct.)

```
student@okan-lab:~/lab$ cut -d' ' -f9 logs/access.log | sort | uniq -c | sort -rn
    148 200
     57 404
     34 304
     24 500
     20 301
     17 403
```

```
student@okan-lab:~/lab$ tail -n +2 data/students.csv | cut -d, -f3 | sort | uniq -c | sort -rn
     17 ECT
     11 ME
     10 CS
      2 EE
```

Part F

```
#!/bin/bash
# count_lines.sh -- print the number of lines of every regular file in a directory
# usage: ./count_lines.sh DIRECTORY
if [ $# -ne 1 ] || [ ! -d "$1" ]; then
    echo "usage: $0 DIRECTORY" >&2
    exit 1
fi
total=0
for f in "$1"/*; do
    if [ -f "$f" ]; then
        n=$(wc -l < "$f")
        echo "$f: $n lines"
        total=$((total + n))
    fi
done
echo "total: $total lines"
```

```
student@okan-lab:~/lab$ chmod +x scripts/count_lines.sh
student@okan-lab:~/lab$ ./scripts/count_lines.sh logs
logs/access.log: 300 lines
logs/system.log: 120 lines
total: 420 lines
student@okan-lab:~/lab$ ./scripts/count_lines.sh data
data/cities.txt: 30 lines
data/numbers.txt: 20 lines
data/students.csv: 41 lines
total: 91 lines
student@okan-lab:~/lab$ ./scripts/count_lines.sh nosuchdir
usage: ./scripts/count_lines.sh DIRECTORY
student@okan-lab:~/lab$ echo $?
1
```

Grading guide: argument and directory check with usage on stderr and `exit 1` (3 pt); loop over `"$1"/*` with `-f` test (2); line count with `wc -l <` (2); total with `$(( ))` and output (2).

Extra drill (not part of the 100 points)

1. A program calls `fork()` in a loop four times. How many processes exist at the end? *Answer:* $2^4 = 16$, of which 15 were created by `fork()`.
2. Amdahl, $S = 0.1$, $N = 8$. *Answer:* $1/(0.1 + 0.9/8) = 1/0.2125 \approx 4.71$; limit $1/S = 10$.
3. $U(n) = 1 - 0.75^n$: $U(2)$, $U(4)$ and the smallest n with $U \geq 90\,\%$. *Answer:* 0.4375, 0.684; $n = 9$ ($U = 0.925$; $n = 8$ gives only 0.8999).
4. A counting semaphore starts at $S = 2$; five threads call `wait(S)`. *Answer:* two pass (S becomes 0), three block until other threads call `signal(S)`.
5. Exponential averaging with $\alpha = 0.5$, $\tau_0 = 12$ ms and bursts 8, 4, 10 ms. *Answer:* $\tau_1 = 10$, $\tau_2 = 7$, $\tau_3 = 8.5$ ms.

9 Course Wrap-up and Where to Go Next

The course in eight sentences

1. An OS is a resource manager and an extended machine; dual mode and system calls protect it (Weeks 1–2).
2. Processes move between five states; the PCB and the context switch make multitasking possible (Week 3).
3. Threads and schedulers share the CPU: FCFS, SJF/SRTF, Priority and RR, judged by waiting and response time (Weeks 4–5).
4. Shared data needs mutual exclusion: mutex locks and semaphores (Week 5).
5. Memory is translated by the MMU: paging, TLB, virtual memory, page replacement (Weeks 6–7).
6. Files are inodes with names in directories; links give a file several names (Weeks 7 and 10).
7. The shell turns commands into processes and system calls: navigation, files, permissions, pipes, jobs (Weeks 9–13).
8. Scripts automate everything you can type (Week 14).

The theory explains why the commands behave as they do: `ps` shows the process states of Week 3, `ls -i` the inodes of Week 7, `chmod` the protection that Week 2 motivated with dual mode, and every pipe is two processes connected by the kernel. Keep your `~/lab` folder and these notes: they are a reference you can use long after the exam.

Keep practising

- Install Linux in a virtual machine or under Windows with WSL (`wsl --install`) and use it daily; a Raspberry Pi is a cheap Linux lab.
- OverTheWire *Bandit*, a free game that teaches the shell level by level: `https://overthewire.org/wargames/bandit/`

- MIT, *The Missing Semester of Your CS Education* (shell, scripting, tools): https://missing.csail.mit.edu/
- https://explainshell.com/ explains any command line; https://www.shellcheck.net/ finds bugs in scripts.

Certifications (optional)

LPI *Linux Essentials* is a good first goal; CompTIA *Linux+* or LPI *LPIC-1* are the next step; later Red Hat *RHCSA* or the Linux Foundation's *LFCS*. Exam objectives and fees change: check the official sites before you register.

Further reading

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed., Wiley: Chapters 1–2 (introduction, OS structures), 3 (processes), 4 (threads), 5 (CPU scheduling), 6 (synchronization tools), 9 (main memory), 10 (virtual memory), 13 (file-system interface).
- R. H. and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (free online), for example:
  - Chapter 4, "The Abstraction: The Process": https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-intro.pdf
  - Chapter 7, "Scheduling: Introduction": https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched.pdf
  - Chapter 18, "Paging: Introduction": https://pages.cs.wisc.edu/~remzi/OSTEP/vm-paging.pdf
  - Chapter 19, "Paging: Faster Translations (TLBs)": https://pages.cs.wisc.edu/~remzi/OSTEP/vm-tlbs.pdf
  - Chapter 22, "Beyond Physical Memory: Policies": https://pages.cs.wisc.edu/~remzi/OSTEP/vm-beyondphys-policy.pdf
  - Chapter 39, "Interlude: Files and Directories": https://pages.cs.wisc.edu/~remzi/OSTEP/file-intro.pdf
- W. E. Shotts, *The Linux Command Line*, No Starch Press (free PDF from the author: https://linuxcommand.org/tlcl.php).

Good luck in the final exam — and thank you for a great semester. Questions before the exam: ask on the course Telegram channel [office hours before the exam: to be announced].